

Unitary transformations of temporal pulsed modes via temporal trapping

Where the project stands, what we learned, and where to go next

Lukass Kellijs

with Ruidi Zhu and Prof. Logan G. Wright

September 2026 • covers work from December 2024 to April 2026

At a glance

The idea. Light pulses that travel together through an optical fiber influence each other: a strong pulse changes the refractive index that a weak pulse experiences (cross-phase modulation). We use this to control a weak *signal* pulse with a strong *trap* pulse. The goal is to shape the trap so that the fiber applies a chosen *unitary transformation* to the signal — a programmable, energy-conserving operation on light, useful for photonic computing and quantum optics. We find the right trap by simulating the fiber and optimizing the trap with gradient descent.

What exists now. A differentiable fiber simulator and a training pipeline, organized as a small Python library ([nlse/](#)) with the experiments in [approaches/](#), all in the repository github.com/lukass16/LGW-Lab-NLSE. About 50 documented training runs, and a first model of a ring-resonator version.

General lessons so far.

- **One trap can steer many signal modes at once**, but getting the *phase* right, and not only the intensity, is the real challenge.
- **What you measure is what you get.** The loss function decides what the optimizer learns; losses that ignore phase give results that look good but are not unitaries.
- **Larger transformations are harder.** Unitaries on a few modes are reproduced well; larger ones are not yet.
- **Numerics matter.** Every result rests on an accurate solver. Step sizes and grids must be checked carefully, especially for strong traps and high-order modes (solver: [nlse/solvers.py](#); worked examples: [examples/](#)).

Main open questions. How should the trap pulse be parametrized, and can the parametrization itself be optimized? Is the difficulty with larger transformations physical, or an optimization problem? Can a ring resonator do better than a straight fiber?

If you are picking this up: read Sections 2 and 3, then Section 8 (open questions) and Section 9 (how to run things). The complete timeline and every training run are in Appendices A and B.

Contents

1	Motivation	3
2	The physical picture	4
2.1	Pulses as vectors: the Hermite–Gauss basis	4
2.2	Two pulses in a fiber	5
2.3	The strong–weak limit: a Schrödinger equation with a programmable potential	5
3	How the inverse-design problem is set up	6
3.1	Loss functions: what the optimizer minimizes	7
3.2	Evaluation metrics: how runs are compared	7
4	Numerics and shaping a single pulse (Oct–Nov 2025)	9
4.1	Getting the numerics right	9
4.2	First inverse design: shaping one pulse	9
5	Learning unitaries in a fiber (Nov 2025 – Apr 2026)	11
5.1	One trap for 16 modes — right intensities, wrong phases	11
5.2	Loss functions matter	11
5.3	Making the strict loss train	15
5.4	How performance scales with the number of modes	16
5.5	Side track: the scanning approach (dropped)	17
6	The ring resonator (Feb – Apr 2026)	17
7	Status at a glance	20
8	Open questions and potential ideas	20
9	Hand-off: how to pick this up	21
A	Timeline	23
B	Every training run	24
C	Default parameters	26

1 Motivation

Unitaries are the building blocks. A *unitary transformation* is a linear map U that conserves energy, $U^\dagger U = I$. A device that could apply *any* unitary, reprogrammably and to many modes at once, would be a general tool for photonic computing, for emulating light–matter interactions, and for building quantum circuits out of multimode light.

Existing approaches have hard limits.

- *Classical*: Onodera, Stein *et al.* [1] used a 2D programmable electro-optic waveguide to realize arbitrary passive linear circuits. It has many tunable parameters, but the index modulation is shallow, the device is short, and loss is high.
- *Quantum*: Presutti, Wright *et al.* [2] used adiabatic frequency conversion of broadband squeezed light to program spectral correlations. Here phase matching, pump shaping and a custom nonlinear crystal limit accessibility and programmability.

Our idea: temporal trapping by cross-phase modulation. In a Kerr medium such as silica fiber, the refractive index grows with the local light intensity. When two pulses travel together, each one's intensity changes the index that the *other* one sees — this is cross-phase modulation (XPM). If one pulse is much stronger, it writes an index pattern that travels with the weak pulse and acts like a potential well for it: a *temporal trap*. Shaping the trap reshapes the potential, and with it the transformation the weak pulse undergoes (Figure 1). This is attractive because

- it runs in **standard fiber**: mature, ultra-low loss, and long, so small effects can accumulate;
- **no phase matching** is needed, and the device is programmed simply by shaping a pulse;
- the induced index change can be **large**, and in principle both classical and quantum properties of the signal can be controlled.

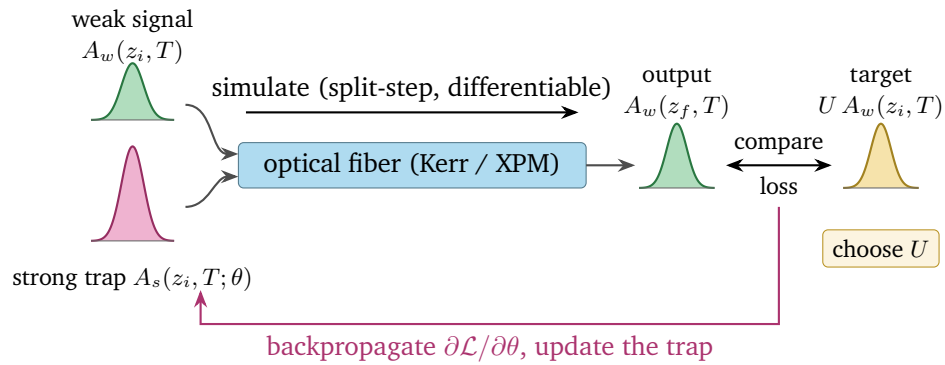


Figure 1: The inverse-design loop. A weak signal and a strong trap pulse (parameters θ) propagate together through a fiber. The output signal is compared with the target U applied to the input, and the gradient of the loss is used to update the trap. The goal is *one* trap θ that implements U for *every* input signal.

Project plan and where we are. The original plan (abstract, Sep 2025) had four stages: (1) build an automatically differentiable 1+1D coupled nonlinear Schrödinger equation (NLSE) simulator and use it to shape a pulse; (2) learn unitary transformations in a 1D waveguide (fiber) and find

the limits of the approach; (3) model a micro-ring resonator; (4) later, 3+1D simulations and the quantum regime. Stage 1 is done, stage 2 is the bulk of this report, stage 3 has started, stage 4 has not. The project grew out of an earlier line of work (Spring 2025) on enhancing Kerr nonlinearity with Gaussian operations; see Appendix A.

2 The physical picture

2.1 Pulses as vectors: the Hermite–Gauss basis

We describe a pulse by its slowly varying complex envelope $A(T)$, where T is time measured in a frame that moves with the pulse. To talk about “applying a matrix” to a pulse we expand it in Hermite–Gauss (HG) functions,

$$\psi_n(T; t_0) = \frac{1}{\sqrt{2^n n!} \sqrt{\pi} t_0} H_n\left(\frac{T}{t_0}\right) e^{-T^2/2t_0^2}, \quad (1)$$

$$A(T) = \sum_n a_n \psi_n(T), \quad a_n = \int \psi_n(T) A(T) dT, \quad (2)$$

where H_n are Hermite polynomials. The HG functions are orthonormal, smooth and localized, and they are the standard temporal modes of pulsed quantum optics. Keeping the first B coefficients turns a pulse into a vector $\mathbf{a} \in \mathbb{C}^B$, and a unitary on the pulse is simply $\mathbf{a} \mapsto U\mathbf{a}$ (Figure 2). The pulse energy $\sum_n |a_n|^2$ is conserved. Throughout the project $t_0 = \tau = 0.1$ (simulation units), the same width as the trap soliton below. In the code, the basis is built by `get_hg_basis` and pulses are converted with `time_to_hg / hg_to_time` (`nlse/modes.py`).

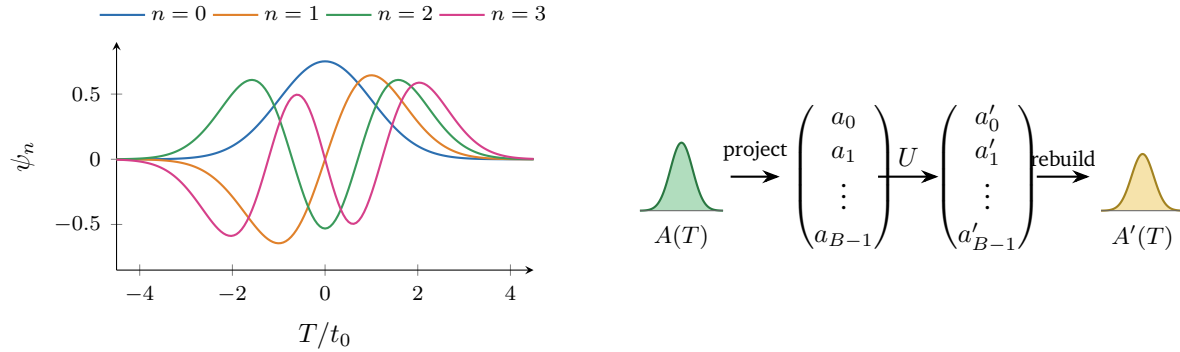


Figure 2: Pulses as vectors. Left: the first four HG modes. Right: a pulse is projected onto the first B HG modes, the coefficient vector is multiplied by U , and the result is turned back into a pulse.

2.2 Two pulses in a fiber

Two pulses A_1 and A_2 co-propagating along a lossless fiber obey the coupled NLSE [3]

$$\frac{\partial A_1}{\partial z} = \left[\underbrace{-\frac{i\beta_{21}}{2} \frac{\partial^2}{\partial T^2}}_{\text{GVD}} + i\gamma_1 \left(\underbrace{|A_1|^2}_{\text{SPM}} + 2\underbrace{|A_2|^2}_{\text{XPM}} \right) \right] A_1, \quad (3)$$

$$\frac{\partial A_2}{\partial z} = \left[\underbrace{-\Delta\beta \frac{\partial}{\partial T}}_{\text{GVM}} - \frac{i\beta_{22}}{2} \frac{\partial^2}{\partial T^2} + i\gamma_2 (|A_2|^2 + 2|A_1|^2) \right] A_2, \quad (4)$$

with $T = t - z/v_{g1}$ and $\Delta\beta = (v_{g1} - v_{g2})/(v_{g1}v_{g2})$. In words:

- **GVD** (group-velocity dispersion, β_2): the group velocity depends on frequency, so the frequency components of a pulse separate as it propagates. The pulse broadens and acquires a chirp.
- **SPM** (self-phase modulation, γ): the pulse's own intensity changes the refractive index it experiences, which adds an intensity-dependent phase to the pulse.
- **XPM** (cross-phase modulation): the intensity of the *other* pulse changes the refractive index as well. For co-polarized pulses this contribution is twice as large as SPM (the factor 2 above).
- **GVM** (group-velocity mismatch, $\Delta\beta$): if the two pulses have different group velocities, they walk off relative to each other. All training so far uses $\Delta\beta = 0$ and $\beta_{21} = \beta_{22} = \beta_2$, $\gamma_1 = \gamma_2 = \gamma$.

The corresponding solvers are in `nlse/solvers.py` (`split_step_fourier_xpm`, its batched version, and `drift_split_step_fourier_xpm` for $\Delta\beta \neq 0$).

2.3 The strong-weak limit: a Schrödinger equation with a programmable potential

Call the weak pulse A_w and the strong one A_s . When $|A_w| \ll |A_s|$ we can drop terms quadratic in A_w :

$$\partial_z A_s = -\frac{i\beta_2}{2} \partial_T^2 A_s + i\gamma |A_s|^2 A_s, \quad (5)$$

$$\partial_z A_w = -\frac{i\beta_2}{2} \partial_T^2 A_w + 2i\gamma |A_s(z, T)|^2 A_w. \quad (6)$$

The strong pulse evolves on its own (Equation (5)), while the weak pulse obeys an equation that is *linear* in A_w . For anomalous dispersion ($\beta_2 < 0$, used throughout) Equation (6) can be written as

$$i \partial_z A_w = \left[-\frac{|\beta_2|}{2} \partial_T^2 + V(z, T) \right] A_w, \quad V(z, T) = -2\gamma |A_s(z, T)|^2. \quad (7)$$

This is exactly the Schrödinger equation for a particle in a potential: z plays the role of time, T of position, $|\beta_2|$ of $1/\text{mass}$, and the strong pulse digs a potential well V — the temporal trap. Two things follow.

1. **Every trap gives a unitary.** The operator in brackets is Hermitian, so propagation from z_i to z_f is a unitary map, $A_w(z_f) = \mathcal{U}[A_s] A_w(z_i)$. The question is not *whether* the trap applies a unitary, but *which* one, and whether we can steer it onto a target.
2. **We control the potential only indirectly.** We choose the trap at the input, $A_s(z_i, T; \theta)$; dispersion and SPM (Equation (5)) then decide how V evolves along the fiber.

The soliton trap — our starting point. For $\beta_2 < 0$, Equation (5) has the fundamental soliton, which keeps its shape and creates a stationary well:

$$A_s = \sqrt{\frac{|\beta_2|}{\gamma\tau^2}} \operatorname{sech}\left(\frac{T}{\tau}\right), \quad V = -\frac{2|\beta_2|}{\tau^2} \operatorname{sech}^2\left(\frac{T}{\tau}\right). \quad (8)$$

Every optimization starts from this trap (`strong_soliton` in `nlse/pulses.py`).

3 How the inverse-design problem is set up

The training code is `approaches/xpm_unitary/train.py`; everything it needs from the physics lives in the `nlse/` library. Figure 1 shows the loop; this section describes each piece.

Simulator. Equations (3) and (4) are solved with the *split-step Fourier method* (`nlse/solvers.py`). Over a short step Δz , dispersion and the nonlinearity are each easy to apply exactly on their own: dispersion is a multiplication by the phase $e^{i\beta_2\omega^2\Delta z/2}$ in the frequency domain, and the nonlinear (SPM and XPM) phase is a multiplication in the time domain. The method alternates between the two, switching domains with fast Fourier transforms. Because the two operations do not commute, treating them one after the other introduces a small error in every step. *Strang splitting* reduces this error by making each step symmetric: half a nonlinear step, a full dispersion step, then the other half of the nonlinear step. The error per step drops from order Δz^2 to Δz^3 , so the method becomes second-order accurate. We compared both variants against a very finely resolved reference and adopted Strang splitting (Section 4.1; `strangsplitting=True` is the default). All operations are PyTorch tensor operations, so autograd gives the exact gradient of the loss with respect to the trap parameters through the whole propagation, on GPU.

What is trained. Only the trap input. It is written as $A_s(z_i, T; \theta) = \sum_{m < M} \theta_m \psi_m(T; \tau)$ with $M = 500$ complex coefficients θ_m , initialized to the soliton. The fiber itself is fixed. We chose $M = 500$ so that the trap has clearly more free parameters than the target: a $B \times B$ unitary has B^2 real parameters (256 for $B = 16$), while 500 complex coefficients give 1000 real ones. Beyond satisfying $M > B^2$, the exact value is an arbitrary choice. Having more parameters than the target does not by itself guarantee that every unitary can be reached; how best to describe the trap is open question Q1 (Section 8).

Training data. The inputs are the first B HG modes (scaled by 0.1 so they are weak). All B inputs are propagated in parallel with the *same* trap (`split_step_fourier_xpm_batch`). This is why the code calls `B batch_size`; it is *not* a batch size in the usual machine-learning sense. Each output is projected back onto the first B HG modes, giving row i of a $B \times B$ matrix V — the transformation the fiber actually realized. The target is U , generated by `approaches/xpm_unitary/transformations.py` (identity, permutation, block rotations, block beamsplitters, or a random “arbitrary” unitary). Energy leaking into modes $\geq B$ is lost from V and so counts as error.

Keeping the trap inside the simulation window. A penalty on the trap intensity outside the central 80% of the time window is evaluated at $m = 8$ positions along z and added to the loss with weight w_{pen} (the alternative, a hard mask, is discussed in Section 4.2).

3.1 Loss functions: what the optimizer minimizes

A loss function is the quantity the optimizer minimizes. It has to be differentiable, and different losses emphasize different aspects of $V \approx U$. Choosing between them turned out to be one of the central questions of the project. Table 1 lists the losses that compare V with U , from weakest to strictest. All are in `nlse/losses.py` and selected with `loss_fn` in the config. The earliest runs used a time-domain loss on output intensities instead (`basic`), which, like `hg`, sees no phase at all.

Name (in code)	Definition	What it can see	What it cannot see
<code>hg</code>	$\frac{1}{B^2} \sum_{i,j} (V_{ij} ^2 - U_{ij} ^2)^2$	Where the energy goes: the power $ V_{ij} ^2$ in every element.	All phases.
<code>fid_energy</code>	$-\frac{1}{B} \sum_i \left(\sum_j V_{ij} U_{ij} \right)^2$	How well the magnitude profile of each row overlaps the target's.	All phases.
<code>fid_phase</code>	$-\frac{1}{B} \sum_i \langle V_i, U_i \rangle ^2$	Relative phases between the entries of one row.	A separate phase on each row. For diagonal or real targets (e.g. identity) this means all phases.
<code>trace</code>	$-\left \frac{1}{B} \text{Tr}(V^\dagger U) \right ^2$	Relative phases between rows as well.	One global phase.
<code>hg_phase</code>	$\frac{1}{B^2} \ V - U\ _F^2$	Every element, magnitude and phase (the Frobenius distance).	Nothing, not even the global phase.

Table 1: Loss functions (`nlse/losses.py`), ordered by strictness. V_i and U_i are the i -th rows (output coefficient vectors), and $\langle V_i, U_i \rangle = \sum_j V_{ij}^* U_{ij}$. Fidelity-type losses only become phase-sensitive through *cross terms*: for a row with a single non-zero entry, $|\langle V_i, U_i \rangle|^2$ does not care about the phase of V_{ii} at all.

3.2 Evaluation metrics: how runs are compared

Because each loss rewards something different, the value of the loss cannot be used to compare runs that were trained with different losses. From February 2026 on, every run therefore also logs three fixed *evaluation metrics*. They are computed on the same V and U at every step, independently of the loss used for training (`eval_*` functions in `nlse/losses.py`). Each one is a normalized version of one of the losses, chosen so that together they range from lenient to strict (Table 2).

Why these three. The trace fidelity is the standard way to compare two unitaries (the normalized Hilbert–Schmidt overlap), and the Frobenius distance is the standard element-wise distance between matrices; the HG fidelity is a per-mode version that is easy to interpret mode by mode. A good result scores well on all three. A result that only gets the intensities right shows up as a high HG fidelity combined with a low trace fidelity and a large ε .

Reading ε . The code logs the Frobenius value $F = \text{mean}_{ij} |V_{ij} - U_{ij}|^2$ (`eval_frobenius_norm`). In this report we quote the relative error ε instead, because it is easier to read: $\varepsilon = 0$ is perfect, and $\varepsilon = 1$ is what an output of *zero* would score, so $\varepsilon > 1$ is worse than no output at all. The two are

Metric	Definition	What it measures	What it ignores	Based on
HG fidelity F_{HG}	$\frac{\sum_i \langle V_i, U_i \rangle }{\sum_i \langle U_i, U_i \rangle } \times 100\%$	How much of each output row overlaps its target row, including the relative phases within a row.	A separate phase on each row; for diagonal or real targets, all phases.	<code>fid_phase</code>
Trace fidelity F_{tr}	$\frac{ \text{Tr}(V^\dagger U) ^2}{ \text{Tr}(U^\dagger U) ^2} \times 100\%$	The overall overlap of the realized and target unitary, including relative phases between rows.	One global phase.	<code>trace</code>
Relative error ε	$\frac{\ V - U\ _F^2}{\ U\ _F^2}$	Element-by-element agreement in magnitude and phase.	Nothing.	<code>hg_phase</code> (Frobenius)

Table 2: Evaluation metrics, from lenient to strict. For the fidelities, 100% is perfect; for ε , 0 is perfect.

related by $\varepsilon = 100 B F$ (the inputs are scaled by 0.1).

4 Numerics and shaping a single pulse (Oct–Nov 2025)

4.1 Getting the numerics right

Every later result depends on the solver being accurate, so it was validated carefully before any training (the solver tests are collected in [examples/02_xpm_solver.ipynb](#); older checks are in [archive/notebooks/](#)). Three lessons shaped every later run (Figure 3).

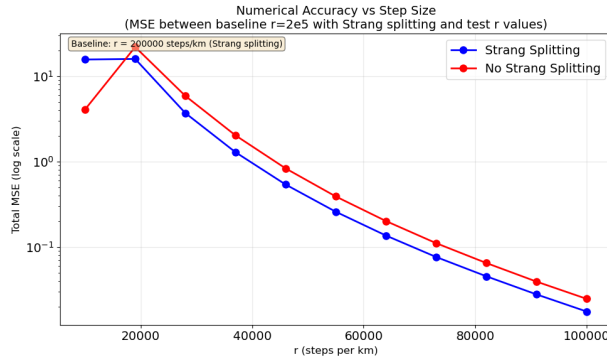
- **Strang splitting converges faster.** We compared plain and Strang splitting (Section 3) against a reference solution computed with a very fine step (2×10^5 steps per km). For the same number of steps, Strang splitting is more accurate (Figure 3a), so it is the default.
- **Strong traps need small steps.** Split-step is accurate only while the nonlinear phase per step $\gamma|A|^2\Delta z$ is small. Scaling the soliton by $\alpha = 10$ with $N_z = 100$ steps gives nonsense; $N_z = 10\,000$ fixes it (Figure 3b,c). In practice $N_z \approx 100\alpha^2$ worked ($\alpha = 5 \rightarrow 2500$, $\alpha = 10 \rightarrow 10\,000$).
- **High-order modes need fine time grids.** HG mode $n = 499$ is visibly under-sampled with $N_t = 4096$ points; $N_t = 16\,384$ resolves it (Figure 3d). High modes also have larger bandwidth, so they disperse much faster.

4.2 First inverse design: shaping one pulse

The first test (Oct 2025) asked a trap to reshape a weak Gaussian into a square (super-Gaussian) pulse ([archive/notebooks/XPM/XPM_1D_Inverse_Design.ipynb](#); single-pulse NLSE examples in [examples/01_nlse_target_pulses.ipynb](#)). It worked (Figure 4) and brought up the question of **keeping the trap inside the simulation window**:

- A **hard mask** zeroes the trap outside a region. With an HG parametrization, however, the modes still extend into the masked region, so the optimizer keeps changing the pulse there, and the pulse can still spread across the whole window if it is too small.
- A **soft penalty** always respects the boundary, but it enforces the same width along the whole fiber, even though pulses naturally spread.
- A hybrid (penalty region wider than the mask) is a possible compromise. We kept the soft penalty.

The trap must dominate. Shrinking the trap relative to the signal from 1000:1 down to 1:1 (at fixed trap energy) makes the square target steadily harder — the final loss grows from 0.08 to 0.32 (Figure 4c).



(a) Error vs. steps per km

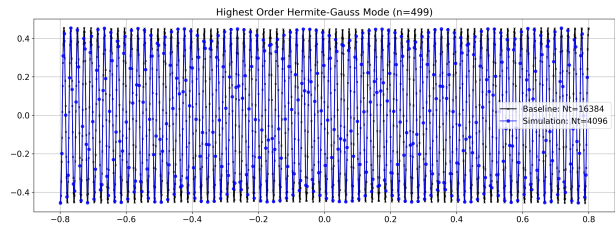
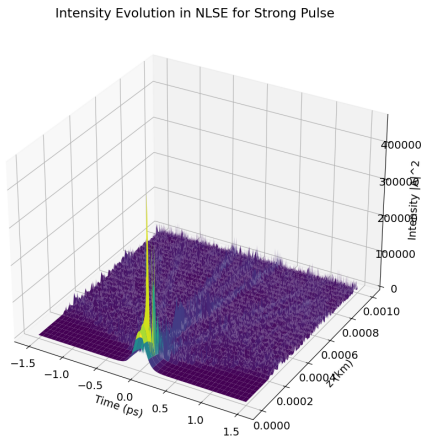
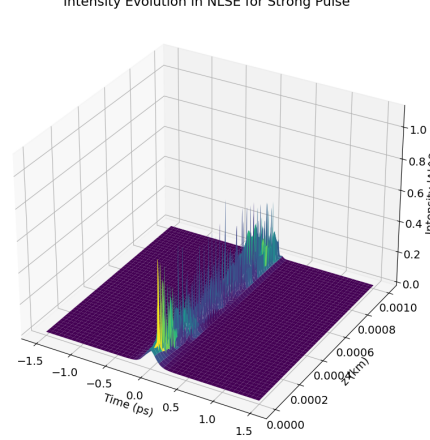
(b) HG mode 499: $N_t = 4096$ vs. 16384(c) Trap $\times 10$, $N_z = 100$: broken(d) Trap $\times 10$, $N_z = 10\,000$: fixed

Figure 3: Numerical lessons. (a) Strang splitting reaches a given accuracy with fewer steps. (b) The time grid must resolve the highest HG mode used for the trap. (c,d) A stronger trap needs proportionally finer steps along z ($N_z \propto \alpha^2$).

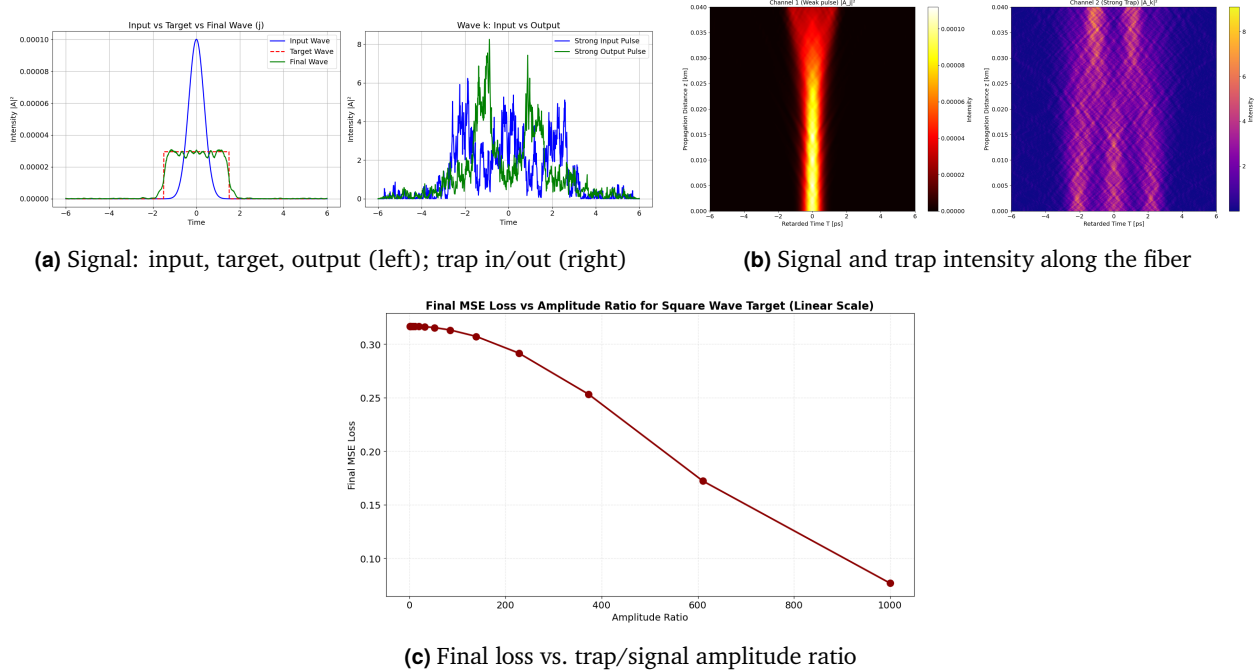


Figure 4: Shaping a single pulse (Oct–Nov 2025). (a,b) A learned trap reshapes a weak Gaussian into a flat-top pulse. (c) The weaker the trap relative to the signal, the worse the result.

5 Learning unitaries in a fiber (Nov 2025 – Apr 2026)

5.1 One trap for 16 modes — right intensities, wrong phases

The next question: *can one trap pulse transform many different inputs correctly at once?* We trained a single trap for $B = 16$ HG inputs (Nov–Dec 2025). With a loss on output *intensities*, the identity worked well: per-mode intensity errors of $\sim 10^{-6}$, dropping to 3×10^{-7} with finer grids (Figure 5a). The learned trap is a complicated, spiky pulse (Figure 5b). Permutations and random “arbitrary” unitaries were partly reproduced (Figure 5c,d).

But these were not unitaries yet. A unitary acts on the complex field, not only on its intensity, so the phase of every output mode has to be right as well. When we looked at the real and imaginary parts of the output (Figure 5e), it became clear that the phase had been essentially ignored: an intensity loss cannot see it, so the optimizer never learned it. This realization made us step back and ask what a good loss function for this problem actually is. It led to the losses and metrics of Sections 3.1 and 3.2 and to the comparison below.

5.2 Loss functions matter

From January to March 2026 we trained with each loss of Table 1 on two targets — the identity and a block-diagonal 50:50 beamsplitter — with $B = 16$ (Adam, learning rate 0.05, 5000 steps; configs in [approaches/xpm_unitary/configs/fiber/](#)), and scored every run with the three metrics of Table 2.

How to read Figure 6. Each group of bars is one training run, labelled by the loss it was trained

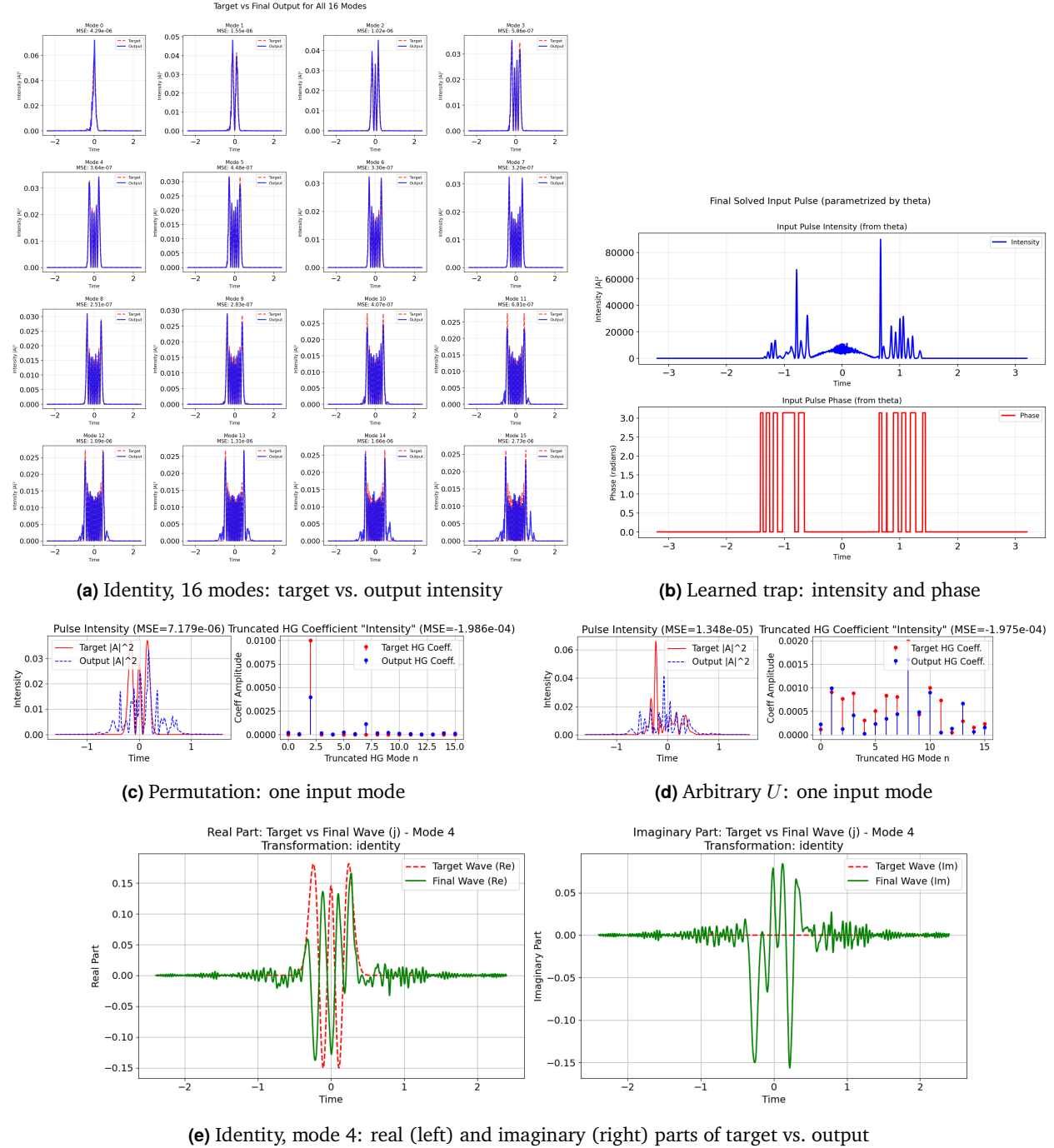


Figure 5: Intensity-only training, $B = 16$ (Nov–Dec 2025). (a) Output intensities match the targets for all 16 modes. (b) The learned trap pulse (intensity and phase). (c,d) Permutations and random unitaries are partly reproduced (red: target HG coefficients, blue: output). (e) The intensity is right but the field is not: the phase is wrong, so this is not yet a unitary.

with (ordered from weakest to strictest). The three panels show the three metrics at the end of training. A perfect run would reach 100% in the first two panels and 0 in the third; the dashed line in the third panel marks $\varepsilon = 1$, the score of an all-zero output.

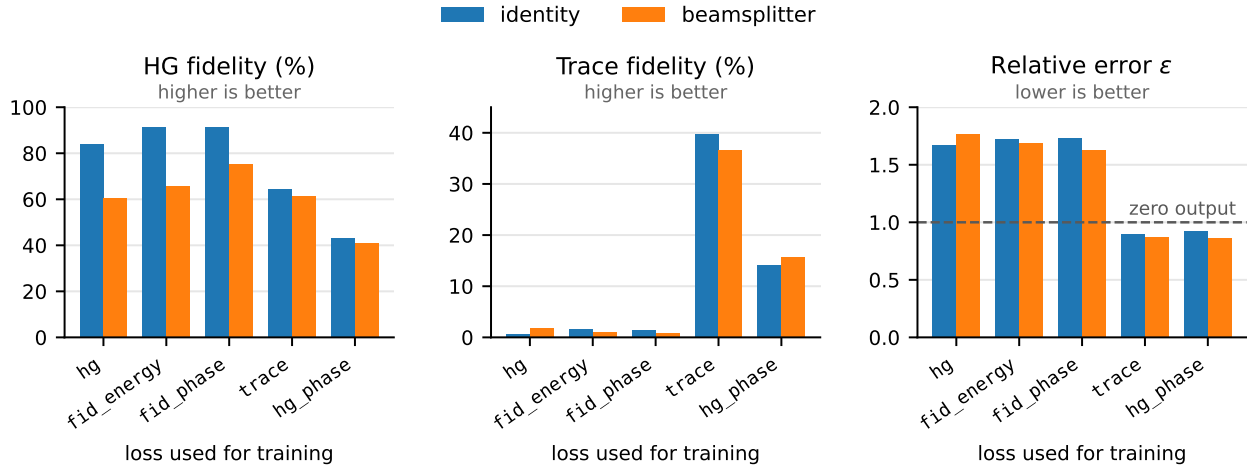


Figure 6: Each loss wins on its own metric ($B = 16$). Phase-blind losses (hg, fid_energy, fid_phase) put the energy in the right modes, giving a high HG fidelity, but with random phases: their trace fidelity is $\approx 1\%$ and $\varepsilon > 1$. Only trace and hg_phase improve the phase-aware metrics, and even they reach only $\approx 40\%$ trace fidelity and $\varepsilon \approx 0.9$.

- **Phase-blind losses look deceptively good.** hg, fid_energy and fid_phase reach 84–91% HG fidelity on the identity, yet their trace fidelity is about 1% and $\varepsilon \approx 1.7$. For the identity, fid_phase and fid_energy are literally the same loss.
- **The trace loss is the most useful single loss.** It gives by far the best trace fidelity ($\approx 40\%$ identity, 37% beamsplitter) and ties for the lowest ε .
- **The global phase is not the whole story.** The trace loss ignores a global phase, so its outputs are rotated as a whole. Removing that rotation lines the phases up much better, but clear errors remain, and they grow for higher modes; the magnitudes also fall off for higher modes (Figure 7).

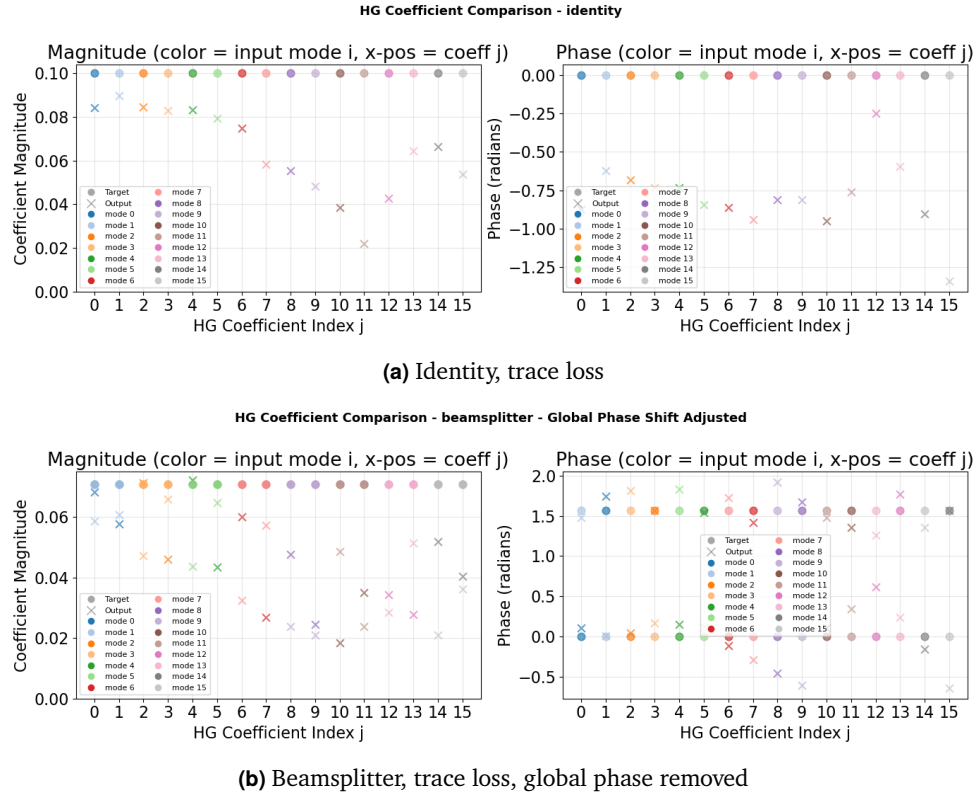


Figure 7: Output HG coefficients (dots: target, crosses: output; color = input mode, x = output coefficient index). Low modes are reproduced well in magnitude; higher modes lose amplitude and phase accuracy.

5.3 Making the strict loss train

The Frobenius loss `hg_phase` is the one we ultimately care about for a faithful unitary, but it trains badly: long plateaus, sudden spikes in the boundary penalty, then small drops (Figure 8). Possible causes we considered: tiny gradients on a flat landscape (Adam’s momentum stalls); competing requirements between modes (a change that helps mode 3 hurts mode 12, so the averaged gradient cancels); and a loss that is simply too strict to start from cold. In March–April 2026 we swept optimizers, learning rates, learning-rate schedules and a two-stage curriculum (Figure 9; configs in `approaches/xpm_unitary/configs/hg-loss/`, options in `make_optimizer` and `make_scheduler` of `approaches/xpm_unitary/train.py`).

What L-BFGS is, and why it fits here. SGD and Adam move along the gradient with a fixed or adaptively scaled step. L-BFGS (limited-memory Broyden–Fletcher–Goldfarb–Shanno) is a *quasi-Newton* method: it uses the last few gradients to build an approximation of the local curvature of the loss, and rescales each step with it; a line search then picks how far to go. This lets it move quickly along flat directions and narrow valleys, where gradient methods crawl. It suits this problem well because the loss is deterministic (every step uses the same B inputs, so there is no sampling noise) and the number of parameters is small (1000 real numbers) — exactly the setting L-BFGS is designed for. We use PyTorch’s `torch.optim.LBFGS` with a strong-Wolfe line search.

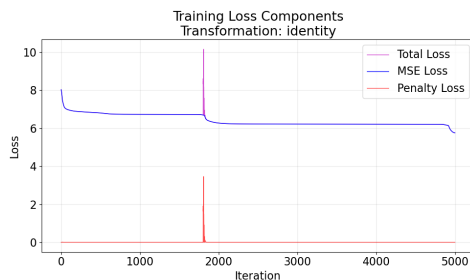


Figure 8: Training the Frobenius loss (identity, $B = 16$, Adam): plateaus, a penalty spike, then a drop.

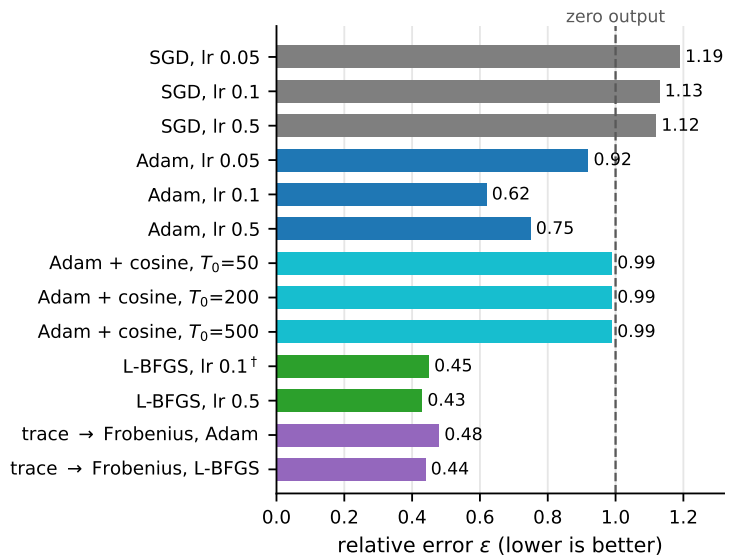


Figure 9: Optimizer sweep (Frobenius loss, identity, $B = 16$, 5000 steps; [†]1000 steps). Lower is better; the dashed line is the score of a zero output.

- **SGD does not move** ($\varepsilon \approx 1.1$ – 1.2 , worse than zero output).
- **Adam depends strongly on the learning rate** (best $\varepsilon = 0.62$ at 0.1). Cosine annealing with warm restarts did *not* help ($\varepsilon \approx 0.99$).
- **L-BFGS is best** ($\varepsilon \approx 0.43$), roughly halving the error of plain Adam.
- **A curriculum helps**: train with the trace loss for 4000 steps, then switch to the Frobenius loss. After only 1000 Frobenius steps it reaches $\varepsilon = 0.48$ (Adam) and 0.44 (L-BFGS) — Adam’s error halves compared with training Frobenius from the start.

The chosen working settings (lab notes, Apr 2026) are **Frobenius loss + L-BFGS** and **trace loss + Adam**, always evaluated on both trace fidelity and relative error.

△ **Note: issue found while writing this report**

The training script keeps the run with the lowest loss as the “best” model, but it does not reset this when the curriculum switches from the trace loss to the Frobenius loss. The two losses live on different scales, so the saved model is always the one from the end of the trace stage, and the saved metrics of the curriculum runs describe that stage. The curriculum numbers above are therefore read from the training log. (Fix: reset `best_loss` at the switch in `train_loop`, [approaches/xpm_unitary/train.py](#).)

5.4 How performance scales with the number of modes

The most important trend so far is how quality falls as the unitary gets bigger. Figure 10 (left) plots the *trace fidelity* — the overall overlap between realized and target unitary, which ignores only a global phase (Table 2) — against the number of modes B , for the identity. Grey is the untrained soliton trap, for reference; blue runs were trained with the trace loss and orange with the Frobenius loss; solid lines used L-BFGS and dotted lines Adam; red crosses mark runs that diverged or timed out. On the right, each pair of panels shows the target $|U|$ and the realized $|V|$ for $B = 4$ and $B = 16$: a perfect result reproduces the bright diagonal.

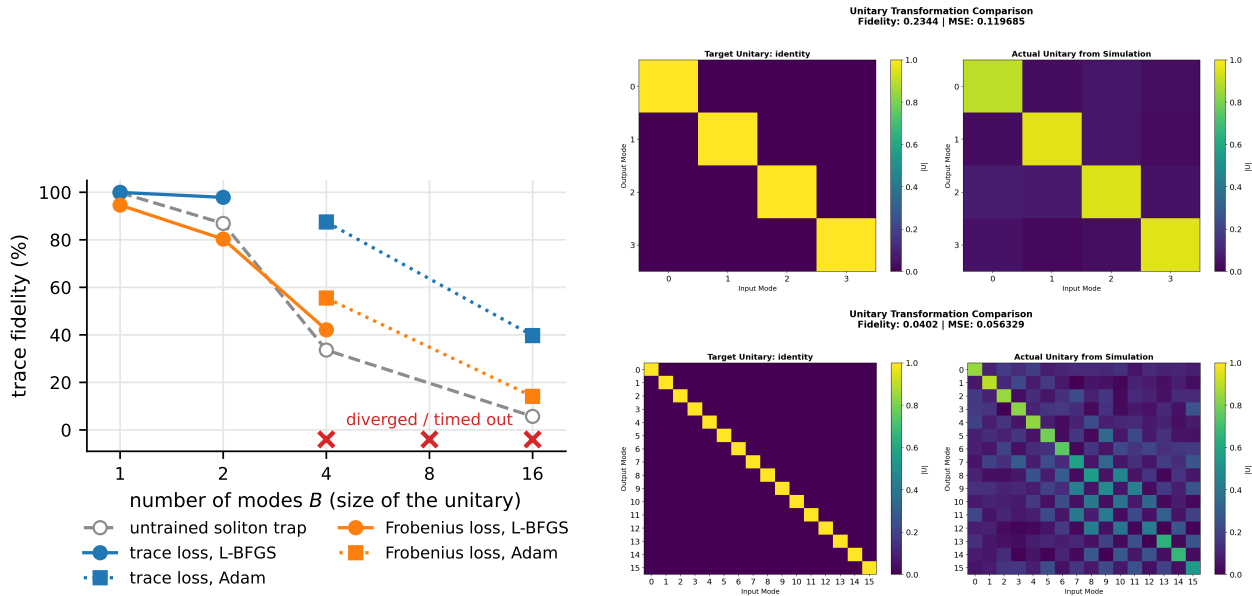


Figure 10: Quality falls with the number of modes. Left: trace fidelity vs. B (identity). The L-BFGS runs used learning rate 0.9 and diverged or timed out for larger B ; fixes are in place but the reruns are still to be done. Right: target and realized $|V|$ for $B = 4$ (top, trace fidelity 87.5%) and $B = 16$ (bottom, 39.7%) — the diagonal fades for higher modes.

- **Training helps a lot where the soliton alone fails:** at $B = 4$ it lifts trace fidelity from 34% to 88%; at $B = 16$ from 6% to 40%.
- **One or two modes are easy:** even the untrained soliton trap reaches 99.9% and 87%.
- **Trace-trained results can have a large ε from the global phase alone:** at $B = 1$ the trace

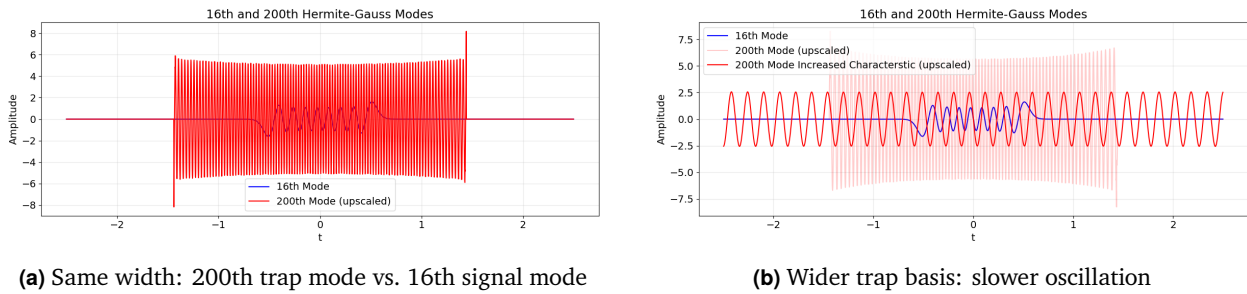
fidelity is 99.95% but $\varepsilon = 3.8$, i.e. the output is right up to an overall sign.

- **The sweep over B is incomplete.** In April 2026 the L-BFGS runs ([approaches/xpm_unitary/configs/analysis/](#)) hit NaNs and time limits for $B \geq 4$. Fixes followed on 21 April (smaller learning rate, strong-Wolfe line search, NaN-safe steps, periodic saving, Adam for the trace runs); the reruns have not been done yet.

5.5 Side track: the scanning approach (dropped)

In December 2025 we tried a *scanning* variant: giving the trap a different group velocity ($\Delta\beta \neq 0$) so that it walks across the signal and different parts of the trap act on the whole signal ([examples/04_xpm_drift.ipynb](#), [archive/scripts/train_xpm_scanning.py](#)). The approach was dropped. The notes do not record the exact reason; the most likely one is that trap components of different widths experience dispersion and self-phase modulation differently, so the trap changes shape as it scans (the notebook already notes that the trap’s own dispersion would need an extra correction factor).

The same work did expose a basic mismatch that is still relevant (Figure 11): with the trap described in HG modes of the same width as the signal, trap modes with $n \gg B$ oscillate far faster than any signal mode and are probably of little use. Describing the trap in a *wider* HG basis avoids this.



(a) Same width: 200th trap mode vs. 16th signal mode

(b) Wider trap basis: slower oscillation

Figure 11: Basis mismatch. With the trap described in HG modes of the same width as the signal, high trap modes oscillate much faster than the signal modes they are supposed to act on (a). A wider characteristic width for the trap basis brings them into a useful range (b). This motivates open question Q1 in Section 8.

6 The ring resonator (Feb – Apr 2026)

Why a ring? In a straight fiber, one trap pulse sets the whole transformation, and only indirectly. In a ring, the signal circulates N times and meets a *fresh* trap pulse on every round trip. That gives up to $N \times M$ parameters and a long interaction length in a compact device (Figure 12).

Model ([approaches/ring_resonator/](#): `train_scaling.py`, `train_arbitrary.py`, `coupler in optics.py`). The ring is coupled to a bus waveguide by a beamsplitter with power reflectivity R ($r = \sqrt{R}$, $t = \sqrt{1 - R}$): the field entering the ring is $ta + rd$ and the field leaving it is $ra + td$, where a is the bus input and d the circulating field. Each round trip propagates length L_z with the coupled NLSE. The weak input is re-injected on every round trip.

How we studied it, and what to expect. Before training, we switched the physical effects on one at a time — dispersion, nonlinearity (SPM), and the trap (XPM) — and tracked the signal energy circulating in the ring after each round trip. The expected behavior is a build-up followed

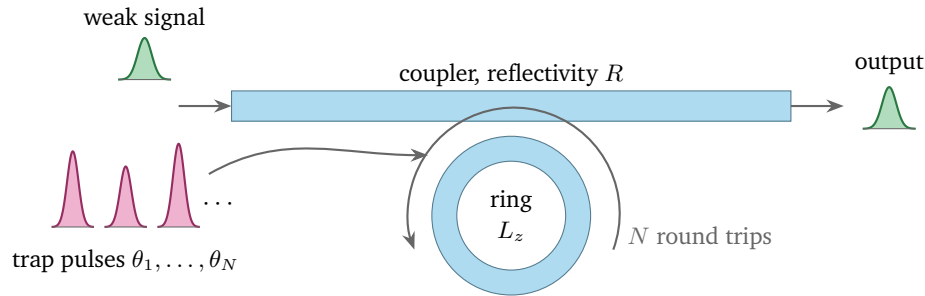


Figure 12: Ring-resonator version. The signal circulates for N round trips and meets a new trap pulse each time. Traps can share one shape with a per-trip amplitude α_k (“scaled”), or each can be independent (“arbitrary”).

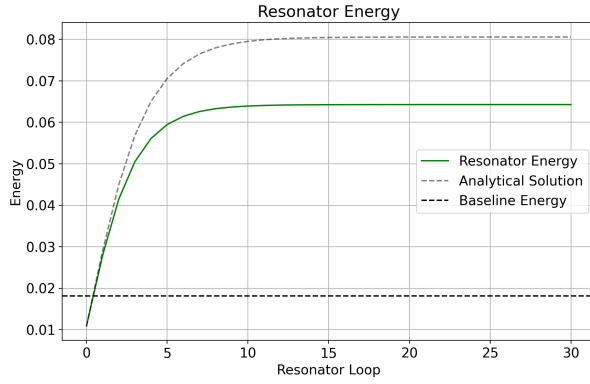
by *saturation*: after enough round trips, the energy coupled into the ring on each trip balances the energy coupled out, and the ring reaches a steady state. For the linear, lossless case this steady state follows analytically from summing the contributions of all previous round trips (dashed curves in Figure 13a–c).

What we learned so far (lab notes of 10 and 23 March 2026; Figure 13):

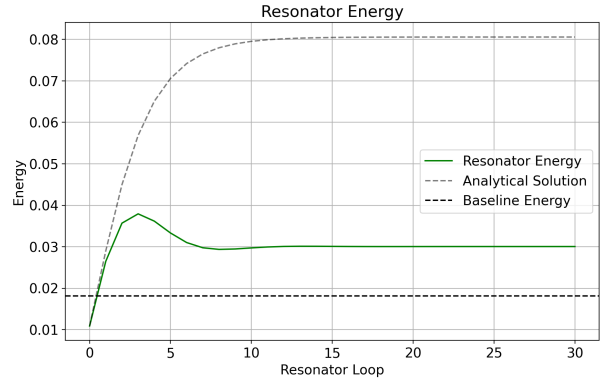
- **The energy saturates as expected** within about ten round trips. With dispersion and SPM only it follows the analytical build-up curve; adding the trap (XPM) lowers the steady-state energy markedly (0.03 vs. 0.064 in the test case: $L_z = 1$, $R = 0.4$, 30 trips).
- **Ring length is a trade-off.** A shorter ring is smaller, faster to simulate, gives more tunable parameters per unit length and a higher steady-state energy — but each trip gives dispersion, SPM and XPM less distance to act.
- **Two trap parametrizations exist:** one shape scaled per trip (`train_scaling.py`, $M + N$ parameters) and fully independent traps (`train_arbitrary.py`, $N \times M$). Both train, but so far only on short runs (1000 steps, loss still falling) with an un-normalized fidelity. No conclusions on performance yet.
- **Main obstacle (Apr 2026):** for short rings, i.e. frequent injection, the freshly injected signal overpowers the effect of the trap, and control becomes impossible. Options noted: inject the signal at a different rate from the traps, only during a finite window, or only every few round trips.

⚠ Warning: check before using the ring trainers

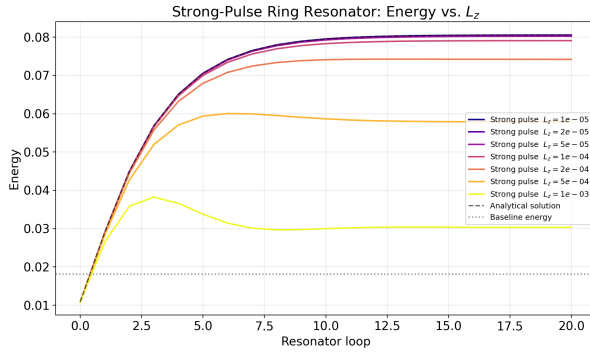
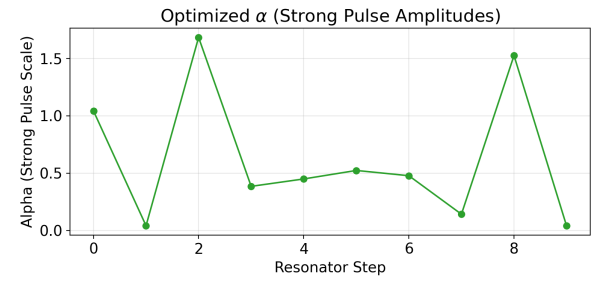
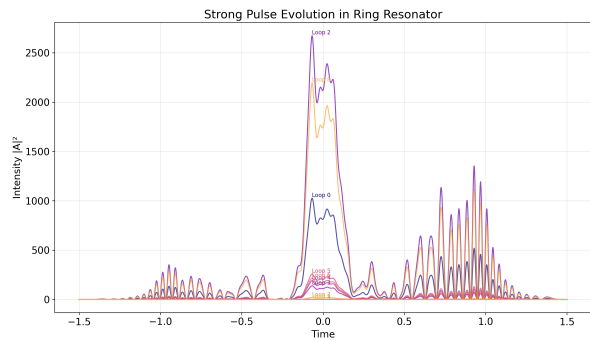
In both ring trainers the ring starts from the chosen HG input mode, but the field re-injected on every round trip is a Gaussian (`weak_input`), not that HG mode ([approaches/ring_resonator/train_scaling.py](#), same in `train_arbitrary.py`). For mode 0 the two differ only in amplitude; for higher modes this is probably not intended.



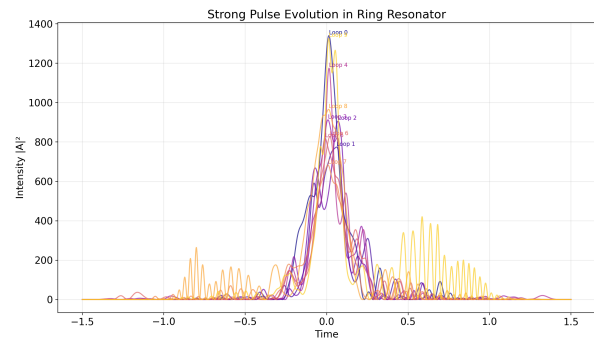
(a) Ring energy per round trip: dispersion + SPM



(b) Ring energy per round trip: + trap (XPM)


 (c) Energy build-up for different ring lengths L_z

 (d) Learned per-trip trap scales α_k


(e) Trap pulses per trip: one scaled shape



(f) Trap pulses per trip: independent

Figure 13: Ring resonator, first results. (a,b) Signal energy in the ring after each round trip (solid), with the analytical build-up (dashed) and the input energy (dotted); the trap lowers the steady state. (c) Build-up for different ring lengths: long rings lose steady-state energy. (d–f) Trap parametrizations after short training runs.

7 Status at a glance

Piece	Status	Notes
Differentiable XPM simulator	done	Strang split-step in PyTorch, GPU; step-size rules known (Section 4.1). <code>nlse/</code>
Training pipeline	done	YAML configs, W&B, checkpoints, SLURM launchers. <code>approaches/xpm_unitary/</code>
Intensity-level control, 16 modes	done	Right intensities, wrong phases — not a unitary (Section 5.1).
Loss / metric study	done	Trace loss most useful; Frobenius strictest (Section 5.2).
Optimizer study (Frobenius)	done	L-BFGS or trace→Frobenius curriculum best, $\varepsilon \approx 0.43$ at $B = 16$ (Section 5.3).
Phase-correct unitaries	partial	$\approx 98\%$ ($B = 2$), $\approx 88\%$ ($B = 4$), $\approx 40\%$ ($B = 16$) trace fidelity.
Scaling with B	partial	$B \leq 4$ only; fixes committed, reruns pending (Section 5.4).
Ring resonator	started	Model and two parametrizations; injection problem identified (Section 6). <code>approaches/ring_resonator/</code>
Trap basis choice	open	Main open question (Q1 in Section 8).
Trap self-evolution (SPM / dispersion off)	open	Configs exist, not yet run (Q2, Section 8).
Realistic units, loss, walk-off	open	Simulation units only; $\Delta\beta = 0$.
Quantum regime, 3+1D	open	Not started.

8 Open questions and potential ideas

The questions below come from our lab notes, meetings and talks. The ideas under each are possible directions, not a fixed plan.

Q1. How should the trap pulse be parametrized?

The trap is currently described by $M = 500$ HG coefficients with the *same* width τ as the signal modes. High trap modes oscillate much faster than the signal modes (Figure 11), need very fine grids, and may mostly make the optimization harder. **Ideas:** (a) *co-optimize the basis and the coefficients*, e.g. make the basis width t_0 (and offset) trainable together with θ — `get_hg_basis` in `nlse/modes.py` already takes both; (b) a *two-step* approach: first choose a good basis (width, number of modes), freeze it, then optimize the coefficients.

Q2. Does the trap's own evolution help or hurt?

Our PI suggested testing how much the trap's own evolution matters. The trap feels its own dispersion and SPM along the fiber, so we only control the potential indirectly. Configs that switch *both* off for the trap already exist (`approaches/xpm_unitary/configs/analysis/trac e-spm-off/`, flag `test_spm_dispersion_off`) but have not been run yet. **Idea:** switch SPM and

dispersion off *separately*, to see which of the two helps or hurts the achievable fidelity.

Q3. Why does quality fall with the number of modes?

Is the drop-off with B (Section 5.4) a physical limit or an optimization problem? **Idea:** rerun the scaling sweep (`approaches/xpm_unitary/configs/analysis/`, launched with `jobs/xpm/launch_analysis.sh`) with the April fixes, and compare all three metrics against B .

Q4. Which loss and optimizer?

The Frobenius loss is the strictest target but the hardest to train; L-BFGS and a trace-first curriculum helped most (Section 5.3). Open points: settle the loss/optimizer pair; fix the curriculum checkpoint (Section 5.3); and check whether the soliton is a poor starting point (an idea from the lab notes).

Q5. How well can a ring resonator do?

Open points from the lab notes and talks: the signal injection schedule (the current blocker, Section 6), the best ring length and total propagation distance, scaled vs. independent traps, and the upper bound on performance compared with the fiber.

Q6. Practical viability and longer term

As planned in the abstract: characterize the practical constraints of the approach (real fiber parameters, loss, walk-off — the drift solver `drift_split_step_fourier_xpm` already exists), and later explore the quantum regime and 3+1D simulations.

9 Hand-off: how to pick this up

Where things are.

- **Code:** github.com/lukass16/LGW-Lab-NLSE (public). The solver library is `nlse/`; the experiments are in `approaches/`. The repository `README.md` and `approaches/xpm_unitary/README.md` summarize setup, commands and losses.
- **Lab notes:** detailed lab notes exist; ask Lukass for access (lukass.kellijs@yale.edu).
- **Experiment tracking:** Weights & Biases, entity `lgwlab`; projects `xpm-identity-operator` (most runs) and `xpm-analysis` (scaling runs).
- **Talks and abstract:** `Lab/Presentations/` (Spring 2025, Fall 2025 $\times 2$, Spring 2026) and `Lab/Abstract/`.

Path (repo root)	What it is
nlse/	The library: split-step solvers (single, XPM, batched, with drift), HG basis tools, pulses, all losses and metrics (<code>losses.py</code>), plotting helpers.
approaches/xpm_unitary/train.py	Main training script: config $\rightarrow$ setup $\rightarrow$ train $\rightarrow$ evaluate $\rightarrow$ save. Optimizers, curriculum, schedules.
approaches/xpm_unitary/analyze.py	Per-mode analysis of a finished run.
approaches/xpm_unitary/configs/	fiber/ (transformations $\times$ losses, Feb–Mar 2026), hg-loss/ (optimizer / schedule / curriculum sweep, Apr 2026), analysis/ (scaling with B).
approaches/ring_resonator/	Batched ring trainers (<code>train_scaling.py</code> , <code>train_arbitrary.py</code>) and their configs.
jobs/{xpm,ring}/	SLURM scripts for Bouchet; <code>launch_analysis*.sh</code> submits one job per config.
examples/, notebooks/	Curated notebooks (solver, XPM, HG basis, identity prototype) and current scratch notebooks.
archive/	Frozen run results (<code>results_full/</code> , one folder per run; see Appendix B), older notebooks, the old monolithic <code>nlse.py</code> , non-batch ring and scanning scripts, and the Spring 2025 Strawberry Fields work.

Running a training job. Create the environment from `environment.yml` (`conda env torch-gpu`) and install the package with `pip install -e ".[train]"`; then, from the repo root:

```
python -m approaches.xpm_unitary.train --config
approaches/xpm_unitary/configs/fiber/config_mini_identity_loss_trace.yaml
```

Options `--loss-fn`, `--transformation`, `--run-dir` and `--no-wandb` override the config. Each run writes `config.yaml`, `losses.json`, `summary.json`, `train.log`, `checkpoints/`, `parameters/` (trap θ , realized and target matrices) and `plots/` into `runs/<name>/`. On the cluster, submit one job per config with

```
bash jobs/xpm/launch_analysis.sh
```

(or the priority variant). Finished runs can be inspected with

```
python -m approaches.xpm_unitary.analyze --run-dir runs/<name>
```

Set `WANDB_API_KEY` in your environment (or pass `--no-wandb`) — never inside a committed script.

Gotchas.

- `batch_size` in configs is B , the size of the unitary — not a batch size.
- The logged Frobenius value is on coefficients scaled by 0.1; convert with $\varepsilon = 100 B F$.
- Trace-trained models can show a huge Frobenius error purely from a global phase.

- The dispersion operator in `nlse/solvers.py` was changed on “10/12/2025” (comment in the code); older notebooks may use the old convention.
- L-BFGS with large learning rates produces NaNs; use the strong-Wolfe line search now in the code.
- Curriculum runs report trace-stage metrics (Section 5.3); the ring trainers re-inject a Gaussian (Section 6).

A good first step is to reproduce `mini-identity-loss-trace` ($B = 4$, $\approx 87\%$ trace fidelity; config `configs/fiber/config_mini_identity_loss_trace.yaml`) to check the setup, before picking up any of the questions in Section 8.

References

- [1] T. Onodera, M. M. Stein, *et al.*, “Scaling on-chip photonic neural processors using arbitrarily programmable wave propagation,” arXiv:2402.17750 (2024).
- [2] F. Presutti, L. G. Wright, *et al.*, “Highly multimode visible squeezed light with programmable spectral correlations through broadband up-conversion,” arXiv:2401.06119 (2024).
- [3] G. P. Agrawal, *Nonlinear Fiber Optics*, 6th ed. (Academic Press, 2019), chapter on cross-phase modulation.
- [4] R. Yanagimoto, T. Onodera, E. Ng, L. G. Wright, P. L. McMahon, H. Mabuchi, “Engineering a Kerr-based deterministic cubic phase gate via Gaussian operations,” *Phys. Rev. Lett.* **124**, 240503 (2020).
- [5] N. Killoran *et al.*, “Strawberry Fields: a software platform for photonic quantum computing,” *Quantum* **3**, 129 (2019).

A Timeline

Built from the git history (118 commits), the lab notes and the talks. “C” is the number of commits that month. Paths refer to the current repository layout.

When	C	What happened	Where
Dec 2024 – Mar 2025	11	Precursor: “Kerr nonlinearity enhanced by multimode squeezing.” Strawberry Fields [5] simulations reproducing a Kerr-based cubic phase gate built from Gaussian operations [4], and squeezing-enhanced Wigner negativity in a Duffing oscillator; grid searches of gate error vs. displacement and squeezing. Hit Fock-cutoff instabilities above ~ 60 and TensorFlow-backend problems. First NLSE inverse design in PyTorch (finding a soliton, square target).	<code>archive/strawberry_fields/</code> , <code>archive/notebooks/NLSE/</code> ; talk: UG Update Spring 2025
Jul 2025	1	README with the goal of the precursor project.	
Sep 2025	0	Pivot to temporal trapping. Ruidi Zhu’s group-meeting talk “Temporal Trapping” (12 Sep); abstract written (19 Sep).	<code>Lab/Abstract/</code>

When	C	What happened	Where
Oct 2025	8	Coupled XPM solver (Strang split-step); weak/strong inverse design with a square target; HG parametrization of the trap; hard vs. soft constraints; super-Gaussian targets.	archive/notebooks/XPM/; talk: UG Update 1 Fall 2025 (27 Oct)
Nov 2025	22	Batch-job training script, SLURM, GPU, W&B. First 16-mode identity with intensity loss (per-mode MSE $10^{-6} \rightarrow 3 \times 10^{-7}$). Amplitude-ratio studies; high-amplitude breakdown and the N_z fix; checkpoints; HG-coefficient losses.	approaches/xpm_unitary/, archive/results_full/16-mode*
Dec 2025	14	Real vs. complex θ ; identity, permutation, arbitrary targets; high-mode validity (N_t , N_z , dispersion); scanning approach.	archive/results_full / (basic-*, hg-*); talk: UG Update 2 Fall 2025
Jan 2026	7	Fidelity losses (energy, phase, phase-adaptive); why fidelity is phase-blind for diagonal/real targets.	archive/notebooks/XPM/
Feb 2026	17	Evaluation metrics (trace, Frobenius, HG fidelity); beamsplitter target; phase-aware loss comparison. Ring-resonator scripts (scaled and arbitrary traps).	archive/results_full / (identity-loss-*, beamsplitter-loss-*)
Mar 2026	12	HG-coefficient plots and global-phase analysis; $B = 4$ “mini” runs; ring: isolated contributions, ring length, energy build-up; loss equations; debugging Frobenius plateaus.	lab-note reports 10.03 and 23.03; archive/results_full/mini-*
Apr 2026	25	Optimizer / learning-rate / cosine-schedule / curriculum sweep; run logging; scaling-with- B configs and SLURM launchers; NaN debugging and fixes (21 Apr); repository reorganization (21 Apr, last commit).	archive/results_full / (hg-phase-*, curriculum-*, analysis-*); talk: UG Update Spring 2026 (14 Apr); lab-note report 21.04
May – Aug 2026	0	No new commits.	
Sep 2026	1	Repository restructured into the nlse library, approaches/ and archive/; this report.	nlse/, approaches/

B Every training run

All runs stored in `archive/results_full/` (formerly `res/`), grouped by study. Metrics are the final values from `losses.json/summary.json`; “–” means the run predates the evaluation metrics (before Feb 2026) or did not finish. HG and trace fidelity in %, ε is the relative Frobenius error. Unless noted, runs use $M = 500$ trap modes, $N_z = 200$, $N_t = 4096$, $L_t = 16$, $L_z = 10^{-3}$. The table is generated from the run folders (`tables/all_runs.tex`).

Run	B Target	Loss	Optim.	lr	Steps	HG Trace ε
<i>Intensity-only training, 16 modes (Nov 2025)</i>						
16-mode	16 identity	time MSE	Adam	1e-1	3000	– –
16-modes-high-n	16 identity	time MSE	Adam	5e-2	3000	– –
16-modes-high-n-high-N	16 identity	time MSE	Adam	5e-2	10000	– –

Run	B	Target	Loss	Optim.	lr	Steps	HG	Trace	ϵ
loss-fn-hg	16	identity	hg (intensity)	Adam	5e-2	10000	–	–	–
loss-fn-normalized-hg	16	identity	norm. hg	Adam	5e-2	10000	–	–	–
<i>Real vs. complex θ; identity / permutation / arbitrary (Dec 2025)</i>									
basic-identity	16	identity	time MSE	Adam	0.1	5000	–	–	–
basic-identity-float	16	identity	time MSE	Adam	5e-2	5000	–	–	– (real θ)
basic-permutation	16	permutation	time MSE	Adam	5e-2	5000	–	–	–
basic-arbitrary	16	arbitrary	time MSE	Adam	5e-2	5000	–	–	–
hg-identity	16	identity	hg (intensity)	Adam	5e-2	5000	–	–	–
hg-permutation	16	permutation	hg (intensity)	Adam	5e-2	5000	–	–	–
hg-arbitrary	16	arbitrary	hg (intensity)	Adam	5e-2	5000	–	–	–
<i>Loss-function comparison, $B = 16$ (Feb 2026)</i>									
identity-loss-hg	16	identity	hg (intensity)	Adam	5e-2	5000	84.1	0.6	1.67
identity-loss-fid-energy	16	identity	fid-energy	Adam	5e-2	5000	91.2	1.6	1.72
identity-loss-fid-phase	16	identity	fid-phase	Adam	5e-2	5000	91.2	1.5	1.73
identity-loss-hg-phase	16	identity	hg-phase	Adam	5e-2	5000	43.2	14.1	0.92
identity-loss-trace	16	identity	trace	Adam	5e-2	5000	64.3	39.7	0.90
beamsplitter-loss-hg	16	beamsplitter	hg (intensity)	Adam	5e-2	5000	60.6	1.9	1.77
beamsplitter-loss-fid-energy	16	beamsplitter	fid-energy	Adam	5e-2	5000	65.7	1.0	1.69
beamsplitter-loss-fid-phase	16	beamsplitter	fid-phase	Adam	5e-2	5000	75.3	0.9	1.63
beamsplitter-loss-hg-phase	16	beamsplitter	hg-phase	Adam	5e-2	5000	41.1	15.6	0.86
beamsplitter-loss-trace	16	beamsplitter	trace	Adam	5e-2	5000	61.5	36.6	0.87
<i>Small unitaries, $B = 4$ (Mar 2026)</i>									
mini-identity-loss-trace	4	identity	trace	Adam	5e-2	5000	93.7	87.5	0.48
mini-identity-loss-hg-phase	4	identity	hg-phase	Adam	5e-2	5000	75.9	55.5	0.15
mini-beamsplitter-loss-trace	4	beamsplitter	trace	Adam	5e-2	5000	93.7	87.7	3.48
mini-beamsplitter-loss-hg-phase	4	beamsplitter	hg-phase	Adam	5e-2	5000	69.7	44.0	0.35
<i>Optimizer / learning-rate / schedule / curriculum sweep, hg-phase loss (Apr 2026)</i>									
hg-phase-sgd-lr5e-2	16	identity	hg-phase	SGD	5e-2	5000	34.0	7.8	1.19
hg-phase-sgd-lr1e-1	16	identity	hg-phase	SGD	1e-1	5000	34.0	8.6	1.13
hg-phase-sgd-lr5e-1	16	identity	hg-phase	SGD	5e-1	5000	34.9	8.4	1.12
hg-phase-adam-lr5e-2	16	identity	hg-phase	Adam	5e-2	5000	43.2	14.1	0.92
hg-phase-adam-lr1e-1	16	identity	hg-phase	Adam	1e-1	5000	53.9	28.2	0.62
hg-phase-adam-lr5e-1	16	identity	hg-phase	Adam	5e-1	5000	46.4	19.9	0.75
hg-phase-adam-cosine-T50-Tm2	16	identity	hg-phase	Adam	5e-2	5000	39.0	11.8	0.99
hg-phase-adam-cosine-T200-Tm2	16	identity	hg-phase	Adam	5e-2	5000	38.0	11.5	0.99
hg-phase-adam-cosine-T500-Tm1	16	identity	hg-phase	Adam	5e-2	5000	39.7	10.8	0.99
hg-phase-lbfgs-lr5e-2	16	identity	hg-phase	L-BFGS	5e-2	–	–	–	– (crashed)
hg-phase-lbfgs-lr1e-1	16	identity	hg-phase	L-BFGS	1e-1	1000	61.0	35.0	0.45
hg-phase-lbfgs-lr5e-1	16	identity	hg-phase	L-BFGS	5e-1	5000	61.4	35.5	0.43
curriculum-adam-trace-adam-hgphase	16	identity	trace→hg-phase	Adam	5e-2	5000	64.3	39.7	0.91 (§5.3)
curriculum-adam-trace-lbfgs-hgphase	16	identity	trace→hg-phase	Adam→L-BFGS	5e-2	5000	64.3	39.7	0.91 (§5.3)
<i>Scaling with number of modes B, L-BFGS (Apr 2026)</i>									
analysis-trace-bs1	1	identity	trace	L-BFGS	0.9	10000	100.0	99.9	3.79
analysis-trace-bs2	2	identity	trace	L-BFGS	0.9	10000	99.1	97.9	2.73
analysis-trace-bs4	4	identity	trace	L-BFGS	0.9	–	–	–	– (NaN)
analysis-trace-bs8	8	identity	trace	L-BFGS	0.9	–	–	–	– (NaN)
analysis-trace-bs16	16	identity	trace	L-BFGS	0.9	–	–	–	– (timeout)
analysis-hg-phase-bs1	1	identity	hg-phase	L-BFGS	0.9	1500	97.3	94.6	0.00
analysis-hg-phase-bs2	2	identity	hg-phase	L-BFGS	0.9	1500	89.7	80.3	0.01
analysis-hg-phase-bs4	4	identity	hg-phase	L-BFGS	0.9	1500	71.3	42.1	0.34
analysis-hg-phase-bs8	8	identity	hg-phase	L-BFGS	0.9	1500	–	–	– (NaN)
analysis-hg-phase-bs16	16	identity	hg-phase	L-BFGS	0.9	–	–	–	– (NaN)

C Default parameters

Symbol / key	Typical value	Meaning
L_z / Lz	10^{-3}	fiber length (simulation units)
N_z / Nz	200	propagation steps (use $\approx 100 \alpha^2$ for a trap scaled by α)
L_t, N_t / Lt, Nt	16, 4096	time window and grid points (16 384 needed to resolve HG mode 499)
β_2 / beta2_j, beta2_k	−10	group-velocity dispersion (anomalous)
γ / gamma_j, gamma_k	1	Kerr nonlinearity
τ / tau	0.1	soliton width and HG basis width t_0
M / N_modes	500	HG modes describing the trap
B / batch_size	1–16	size of the target unitary (number of signal modes)
amplitude_downscale	0.1	signal amplitude relative to unit HG modes
mask_percent, m	0.8, 8	penalty-free fraction of the window; number of z -slices checked
$w_{\text{mse}}, w_{\text{pen}}$	$10^4, 10^3$	loss and penalty weights
lr, N_train	0.05, 5000	learning rate and training steps (Adam default)